

ISPilot Enterprise Architecture & Vertex Agent Engine Guide

Version: 1.0

Status: Production-Ready MVP

Last Updated: August 2026

1. Executive Summary

ISPilot is an enterprise-grade AI platform built to provide natural language access to Intelligent Shelf operational data.

The platform combines:

- Google ADK
- Vertex AI Agent Engine
- Gemini
- BigQuery
- Secret Manager

to deliver business insights and operational recommendations through a specialized multi-agent architecture.

The platform has successfully evolved from a local ADK prototype into a production-ready deployment hosted in Vertex AI Agent Engine.

2. Business Problem

Retail organizations face several challenges:

- Operational data is distributed across multiple systems.
- Business users rely heavily on dashboards and reports.
- Store managers require immediate answers.
- Analysts become operational bottlenecks.

Questions such as:

How is my store performing?

What should I do next?

What stores require attention?

typically require analyst intervention.

ISPilot eliminates this dependency by allowing business users to interact directly with operational data using natural language.

3. Business Vision

ISPilot is not a chatbot.

ISPilot is a business intelligence platform powered by AI.

The objective is to create reusable business capabilities that can be exposed through:

- AI Agents
- APIs
- Teams
- Copilot Studio
- Mobile Apps
- Future Enterprise Applications

4. What Makes ISPilot Different

Traditional Model

```
graph TD; User --> Dashboard; Dashboard --> Analyst; Analyst --> Action;
```

Business users depend on dashboards, reports and analysts.

Insights are delayed.

Actions are reactive.

ISPilot Model

```

User
↓
Coordinator Agent
↓
Specialized Agent
↓
Business Services
↓
BigQuery
↓
Business Answer

```

Benefits:

- Faster decisions
- Reduced analyst dependency
- Consistent recommendations
- Natural language interaction
- Reusable business services

5. Functional Multi-Agent Architecture

```

User
|
▼
Coordinator Agent
|
├── Shelf Analyst
├── Store Coach
├── Root Cause Agent
└── Executive Agent

```

Every agent specializes in a specific business responsibility.

6. Coordinator Agent

Purpose

Coordinator acts as the central orchestrator.

Responsibilities:

- Intent classification

- Agent routing
- Delegation
- Conversation flow management

Coordinator never:

- Queries BigQuery
- Executes business logic
- Generates recommendations

The Coordinator's responsibility is orchestration.

Example

User asks:

How is Talca Colin performing?

Coordinator routes:

Shelf Analyst

User asks:

How can Talca Colin improve?

Coordinator routes:

Store Coach

7. Shelf Analyst

Mission

Answer:

What is happening?

Examples:

```
How is Chile today?  
  
How is Talca Colin performing?  
  
Show the rankings.  
  
What are the trends?  
  
Compare today vs yesterday.
```

Current Tools

```
resolve_store()  
  
get_store_summary()  
  
compare_store_daily()  
  
get_store_trend()  
  
get_store_ranking()
```

Current Capabilities

- Country KPIs
- Store KPIs
- Store Name Resolution
- Store Code Resolution
- Rankings
- Comparisons
- Historical Trends

Representative Questions

```
How is Talca Colin performing?  
  
How is Chile performing?  
  
Which stores are underperforming?  
  
What are the daily rankings?
```

Show the trend for Talca Colin.

8. Store Coach

Mission

Answer:

What should I do?

Examples:

How can Talca Colin improve?

What should the manager do?

What actions should be prioritized?

What opportunities exist?

Current Tools

`diagnose_store()`

`identify_priority_stores()`

`generate_action_plan()`

Current Capabilities

- Recommendations
- Prioritization
- Action Plans
- Coaching Guidance

Store Coach consumes analytical information produced by Shelf Analyst and converts it into operational recommendations.

Representative Questions

How can Talca Colin improve?

What actions should be prioritized?

What should be fixed first?

Generate a weekly action plan.

9. Future Agents

Root Cause Agent

Mission

Answer:

Why is this happening?

Future responsibilities:

- Root Cause Detection
- Classification Analysis
- Operational Categorization
- Failure Drivers

Executive Agent

Mission

Answer:

What does this mean for the business?

Future responsibilities:

- Executive Summaries
- Strategic Insights
- Business Impact Analysis
- Decision Support

10. Architecture Principles

Multi-Agent Architecture

Benefits:

- Separation of concerns
- Scalability
- Maintainability
- Reusability

Business-First AI

Users interact using:

```
SNSG
Quiebre
Bodega
```

instead of:

```
N_ON_SHELF
OOS_STORE
OOS_SHELF
```

No SQL Inside Agents

SQL belongs to:

```
Gateways
Services
```

not agents.

Agents should focus on:

```
Reasoning
Planning
Delegation
Tool Selection
```

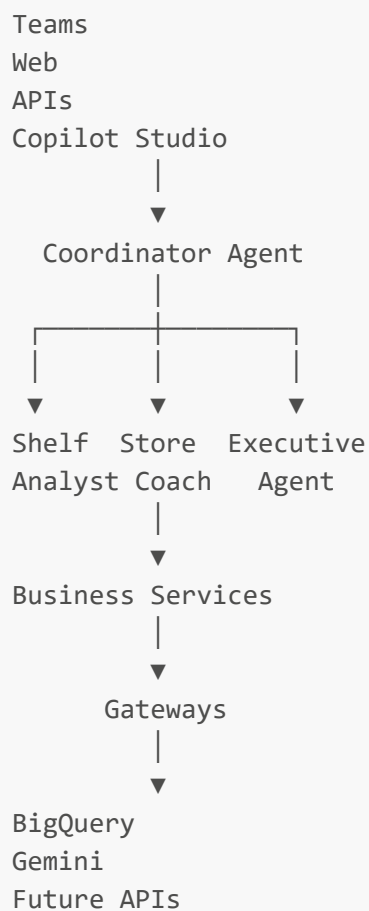
Reusable Business Services

Business intelligence should be reusable.

The same service should support:

- Agents
- APIs
- Dashboards
- Future applications

11. Technical Architecture



12. Codebase Architecture

Agents

Responsibilities:

- Prompting
 - Tool orchestration
 - Agent collaboration
-

Services

Responsibilities:

- Business Logic
- KPI Aggregation
- Analytics

Examples:

```
analytics_service.py
store_service.py
```

Gateways

Responsibilities:

- BigQuery access
- External integrations
- Data retrieval

Examples:

```
nsg_gateway.py
store_gateway.py
```

Repositories

Responsibilities:

- Persistence logic
 - Future abstraction layer
-

Domain

Responsibilities:

- Business entities
 - Domain models
-

- Shared business concepts

Common

Shared components:

```
config.py
bigquery_client.py
gateway_gemini.py
gateway_token.py
```

13. Final Project Structure

```
intelligent-shelf-agents/
├── agents/
│   └── coordinator/
│       ├── agent.py
│       ├── requirements.txt
│       ├── .agent_engine_config.json
│       └── subagents/
│           ├── shelf_analyst/
│           ├── store_coach/
│           ├── executive/
│           └── root_cause/
├── common/
├── services/
├── gateways/
├── repositories/
├── domain/
└── tests/
```

14. Architecture Decision Records

ADR-001

Why Google ADK

Decision:

Use Google ADK as the primary orchestration framework.

Reasons:

- Native Agent support
 - Tool calling
 - Multi-agent architecture
 - Vertex AI integration
-

ADR-002

Why Multi-Agent Architecture

Decision:

Use specialized agents instead of one monolithic agent.

Benefits:

- Maintainability
 - Separation of concerns
 - Scalability
 - Easier evolution
-

ADR-003

Why Coordinator + Subagents

Initial structure:

```
agents/  
├─ coordinator/  
├─ shelf_analyst/  
└─ store_coach/
```

Worked locally.

Failed in Vertex Agent Engine.

Final structure:

```
agents/  
└─ coordinator/  
    └─ subagents/
```

Benefits:

- Compatible with ADK packaging
-

- Simpler deployment
- Better runtime stability

15. BigQuery Architecture

Reporting View

```
vw_nsg_report
```

Provides:

- SNSG
- Quiebre
- Bodega
- Store-level KPIs

Classification View

```
vw_smartnsg_classification_cl_v2
```

Provides:

- Root Cause Classification
- Opportunity Categorization
- Coaching Inputs

16. Security Architecture

Runtime Service Account

```
sa-tot-osa@corp-stro-salesinventory-prod.iam.gserviceaccount.com
```

Secret Manager

Required Secret:

```
genai-gateway-jwt-prod
```

Required Role:

```
roles/secretmanager.secretAccessor
```

BigQuery Roles

```
roles/bigquery.jobUser  
roles/bigquery.dataViewer
```

Service Usage

```
roles/serviceusage.serviceUsageConsumer
```

17. Runtime Configuration

File:

```
agents/coordinator/.agent_engine_config.json
```

```
{  
  "service_account": "sa-tot-osa@corp-stro-salesinventory-  
prod.iam.gserviceaccount.com",  
  "env_vars": {  
    "IS_BQ_PROJECT": "corp-stro-salesinventory-prod",  
    "IS_BQ_DATASET": "acc_tot_cp_is_prd",  
    "IS_NSG_REPORT_VIEW": "vw_nsg_report",  
    "IS_CLASSIFICATION_VIEW": "vw_smartnsg_classification_cl_v2"  
  }  
}
```

18. Local Development

Create Environment

```
python3.12 -m venv .venv
```

Activate

```
source .venv/bin/activate
```

Install Dependencies

```
pip install -e .
```

Run ADK

```
adk web agents/coordinator
```

19. Vertex Deployment

Official Deployment Method

```
adk deploy agent_engine
```

Production Command

```
adk deploy agent_engine \  
  --project=corp-stro-salesinventory-prod \  
  --region=us-central1 \  
  --display_name=intelligent-shelf-coordinator \  
  --extra_packages common \  
  --extra_packages services \  
  --extra_packages gateways \  
  --extra_packages repositories \
```

```
--extra_packages domain \
agents/coordinator
```

20. Deployment Timeline

Stage 1

Local ADK

☒ Working

Stage 2

Vertex Deployment

☒ Missing common

Resolution:

```
extra_packages
```

Stage 3

Vertex Deployment

☒ Missing secretmanager

Resolution:

```
google-cloud-secret-manager
```

Stage 4

Vertex Deployment

☒ No module named agents.shelf_analyst

Resolution:

```
Coordinator-owned subagents
```

Stage 5

Vertex Deployment

✗ Missing IS_CLASSIFICATION_VIEW

Resolution:

Environment configuration

Stage 6

☒ Playground operational

Stage 7

☒ REST API operational

Stage 8

☒ Multi-Agent routing validated

21. End-to-End Request Flow

Example:

How is Talca Colin performing?

Step 1

User sends request.

Step 2

Coordinator classifies intent.

Performance Question

Step 3

Coordinator delegates.

```
transfer_to_agent(  
    "shelf_analyst"  
)
```

Step 4

Store resolution.

```
resolve_store()
```

Step 5

Analytics execution.

```
get_store_summary()  
  
compare_store_daily()
```

Step 6

Business response generated.

22. Vertex Playground Validation

Successfully validated:

- ☒ Coordinator
- ☒ Shelf Analyst
- ☒ Tool Calling
- ☒ BigQuery Access
- ☒ Multi-Agent Execution

☒ Store Resolution

☒ KPI Summaries

23. REST API Validation

Create Session

```
{  
  "classMethod": "create_session"  
}
```

Returns:

```
{  
  "session_id": "..."  
}
```

Query Session

```
{  
  "classMethod": "stream_query"  
}
```

Successfully validated through:

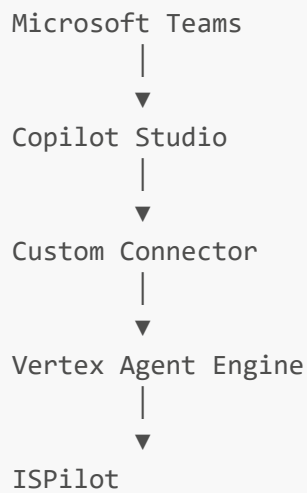
```
Reasoning Engine REST API
```

24. Current Production Runtime

```
projects/390358249123/locations/us-  
central1/reasoningEngines/4655320687131492352
```

25. Teams & Copilot Studio Integration

Target Architecture:



No changes are required inside ISPilot.

Only the connector layer must be implemented.

26. Operational Runbook

Deploy

```
adk deploy agent_engine
```

Logs

```
gcloud logging read ...
```

Playground

```
Vertex AI  
→ Agent Engines  
→ Playground
```

REST Validation

`create_session`

`stream_query`

27. Current State

- ☒ Coordinator Agent
 - ☒ Shelf Analyst
 - ☒ Store Coach
 - ☒ BigQuery Integration
 - ☒ Gemini Integration
 - ☒ Secret Manager Integration
 - ☒ Vertex Agent Engine
 - ☒ REST API
 - ☒ Playground
 - ☒ Multi-Agent Routing
 - ☒ Teams Ready
 - ☒ Copilot Studio Ready
-

28. Lessons Learned

1. Multi-agent architecture provides clear separation of responsibilities.
 2. Coordinator should only orchestrate.
 3. Business services should remain agent-agnostic.
 4. Runtime configuration is critical in Agent Engine.
 5. Vertex Agent Engine can be consumed directly through REST APIs.
 6. Copilot Studio integration does not require changes inside ISPilot.
-

29. Strategic Roadmap

Phase 1

☒ Shelf Analyst

Phase 2

☒ Store Coach

Phase 3

Root Cause Agent

Phase 4

Executive Agent

Phase 5

Recommendation Engine

Phase 6

Microsoft Teams Integration

Phase 7

Enterprise Observability

Phase 8

Enterprise AI Platform Expansion